Developer Guide

A practical manual for working on codeAnalyzer **offline, by hand**. It explains *why* the project is built the way it is, *where* each piece lives, and *which files to touch* for the changes you are most likely to make.

For the high-level data flow and the analyzer JSON contract, see `architecture.md`. This guide is the hands-on companion to it.

---

1. What this tool is (one paragraph)

A **local-only** static analyzer. You give it a folder; it scans supported source files and reports per-method and per-file metrics. For **C#** it finds **TCF methods** (methods whose name starts with a configurable prefix, default `TCF`) and the **helper functions they call; for** C/C++** it reports file-level line metrics only. It runs as a small website on `127.0.0.1`. Nothing ever leaves the machine — no network, no cloud, no AI, no telemetry. Those are hard product constraints, not preferences (see §10).

> **"TCF" is just the prefix.** It used to be "HFS"; it was renamed project-wide. The prefix is a runtime setting (UI field / `DEFAULT_PREFIX`), and the name "TCF" appears in identifiers only as the *internal label for the concept*. If a client uses a different prefix, they type it in the UI — no code change needed.

---

2. Why it is built this way (design decisions)

2.1 Hybrid Python + .NET (Roslyn), not pure Python

The single most important decision. The helper rule requires telling a **project** `Add()` apart from a library `List<T>.Add()`, and resolving a call to the method it actually binds to **across files**. That is *semantic* analysis — it needs a real compiler's symbol resolution.

- **Roslyn** (the actual C# compiler API) does this correctly: it builds a compilation, binds each call to a symbol, and tells us the symbol's source location.
- **tree-sitter / regex / any syntax-only parser was rejected** because it only sees *shapes* of code, not *meaning*. It cannot know that `Add(x)` in one file refers to a method defined in another file, nor distinguish it from an identically named library method. It would force naming heuristics — exactly what the requirements spec forbids.

So: **Python** is the application shell (UI, scanning, line counting, C/C++ comments, reporting, packaging); **C# / Roslyn** is a focused subprocess that answers one question — "for these C# files and this prefix, what are the methods, which are TCF, and which project helpers does each TCF method call?"

tree-sitter remains only a *hypothetical* future option for richer C/C++ parsing; it is **not** used today (the C/C++ side is a small hand-written lexer, §5).

2.2 The two talk over a subprocess + JSON contract

Python invokes the analyzer as a child process, writes a JSON request to its stdin, and reads a JSON response from stdout (see `analyzer_bridge.py` and the C# `Program.cs`). Why a subprocess instead of, say, Python.NET or an embedded runtime:

- Keeps the .NET engine a **self-contained black box** that ships next to the app and needs no .NET install on the user's machine.
- Clean failure isolation: a crash in the analyzer is a non-zero exit + stderr, which Python turns into a friendly error — it can't take down the web app.
- Trivial to test each side independently with synthetic JSON.

2.3 One compilation for all files (cross-file resolution)

The analyzer puts **all** scanned `.cs` files into a *single* `CSharpCompilation`. That is what makes a TCF method in `Service.cs` resolve a helper defined in `Helpers.cs`. Analyzing files one-by-one would lose cross-file calls. (See `ProjectAnalyzer.cs`.)

2.4 No `.csproj` / NuGet restore — BCL references from the runtime

The analyzer references the running runtime's assemblies (`TRUSTED_PLATFORM_ASSEMBLIES`) so it can bind framework calls without a project file or network restore (`ReferenceLoader.cs`). Trade-off: third-party (NuGet) types aren't available, so calls into them don't resolve to a source symbol — which is *fine* for helper-vs-library (they're correctly treated as "library"), but means we don't analyze inside them. This keeps the tool zero-config and offline.

2.5 Client-side pagination, server-rendered shell

The result page can contain thousands of TCF methods / helpers. Sending it all as one giant HTML would be slow. Instead Python embeds the data as JSON `<script>` blobs and a small vanilla-JS paginator renders only the current page (`app.js`). No frameworks, no CDN (local-only): everything is hand-written JS/CSS served from `web/static/`.

2.6 PyInstaller one-dir bundle, self-contained analyzer

Delivery is a folder (`dist/codeanalyzer/`) with `codeanalyzer.exe` + `_internal/`, not a single self-extracting exe. One-dir starts faster and trips antivirus heuristics far less. The Roslyn analyzer is published self-contained per-RID, so the target needs neither Python nor .NET. (See §8 and `packaging/`.)

2.7 Local access token

Because the app listens on a TCP port, another local user (or a malicious web page via the browser) could otherwise poke it. On startup we mint a one-time token, bake it into the URL we open, and bind it to a Strict-SameSite cookie; the Host header is pinned to localhost (anti DNS-rebinding). Disable on a trusted box with `CODEANALYZER_NO_AUTH=1`. (See `web/server.py` `_guard`.)

---

3. Repository map

```
codeAnalyzer/
├── REQUIREMENTS.md            # the requirements spec — the source of truth for behaviour
├── PLAN.md                    # original implementation plan
├── README.md                  # user- and operator-facing readme
├── LICENSE                    # proprietary default (swap if needed)
├── pyproject.toml             # Python package, deps, ruff/mypy/pytest config
│
├── src/codeanalyzer/          # the Python application  (see §4)
│   ├── __main__.py            # entry point: pick port, start Flask, open browser
│   ├── __init__.py            # __version__
│   ├── filetypes.py           # extension → FileKind/Language classification
│   ├── scanner.py             # walk folder, decode bytes, produce SourceFile list
│   ├── line_metrics.py        # the exact line-counting rules
│   ├── cpp_comments.py        # C/C++ comment+string lexer (comment spans)
│   ├── analyzer_bridge.py     # locate + run the .NET analyzer, parse its JSON
│   ├── orchestrator.py        # merge scan + analyzer JSON + metrics → typed result
│   ├── models.py              # the typed result dataclasses
│   ├── report.py              # render_text / render_json / source tree
│   ├── _bundled/analyzer      # the staged analyzer binary (GITIGNORED; built artifact)
│   └── web/
│       ├── server.py          # Flask app: routes, auth guard, request handling
│       ├── templates/         # Jinja2: base / index / results
│       └── static/            # app.js (all UI behaviour) + style.css
│
├── csharp-analyzer/           # the .NET / Roslyn engine  (see §6)
│   ├── TcfAnalyzer.sln
│   ├── src/TcfAnalyzer/
│   │   ├── Program.cs         # stdin→JSON→analyze→stdout
│   │   ├── ProjectAnalyzer.cs # build ONE compilation, analyze each file
│   │   ├── FileAnalyzer.cs    # per-file: methods, comments, diagnostics
│   │   ├── HelperResolver.cs  # *** the helper-detection core ***
```

```
│   │   ├── ComplexityCalculator.cs
│   │   ├── CommentCollector.cs
│   │   ├── ReferenceLoader.cs # BCL references from the runtime
│   │   ├── Models.cs / Json.cs
│   └── tests/TcfAnalyzer.Tests/AnalyzerTests.cs
│
├── tests/python/              # pytest suite (mirrors the Python modules)
├── scripts/                   # build / run / test / package / sample-gen
├── examples/                  # ready-to-analyze sample folders + ground truth
└── docs/                      # architecture.md + this guide
```

The **two files that matter most for correctness** are `csharp-analyzer/src/TcfAnalyzer/HelperResolver.cs` (what counts as a helper) and `src/codeanalyzer/line_metrics.py` (how lines are counted). Treat changes to either with care and back them with tests.

---

4. Python modules, in pipeline order

Read them in this order and the data flow makes sense:

1. **`filetypes.py`** — `classify(path) -> FileKind | None`. Maps extensions (case-insensitive) to a `Language` (`CSHARP` / `C` / `CPP` / header). Unsupported files return `None` and are skipped. *Add a new extension here* (but the requirements spec says don't add file types unless asked).

1. **`scanner.py`** — `scan(folder) -> ScanResult` walks the tree (`os.walk`, sorted for deterministic output), reads bytes, and decodes with a fallback chain (`utf-8` → BOM `utf-8-sig` → `utf-16` → `cp1252` → replacement). Read/permission errors become **warnings**, never exceptions — one bad file can't abort the run. `validate_target` produces the friendly "folder doesn't exist / not a folder" message.

1. **`line_metrics.py`** — the heart of counting. `line_flags(text, comment_spans)` classifies each physical line; `counts_from_flags` tallies them into a `LineCounts` (total / code / comment / inline_comment / blank, plus `comment_ratio`). The required rules are implemented *exactly* here: a line with code **and** a trailing comment increments code, comment, **and** inline_comment. Comment spans come from elsewhere (the analyzer for C#, `cpp_comments` for C/C++) so this module is language-agnostic.

1. **`cpp_comments.py`** — a tiny C/C++ lexer that finds comment spans while correctly skipping string/char literals (so `"// not a comment"` isn't miscounted). Used for C/C++ only; C# comment spans come from Roslyn.

1. **`analyzer_bridge.py`** — `find_analyzer()` locates the binary (explicit arg →

`CODEANALYZER_ANALYZER` env → PyInstaller bundle → installed `_bundled/` → dev `artifacts/`). `run_analyzer(cs_files, prefix)` builds the request, runs the subprocess (600 s timeout), and parses JSON. Any failure → `AnalyzerError`.

1. **`orchestrator.py`** — `analyze(folder, prefix)` ties it together; `build_result(...)` is **pure** (no I/O) so it's unit-testable with synthetic analyzer JSON. It:
    - runs the analyzer over the C# files,
    - for each C# file, turns analyzer JSON into line metrics + `TcfMethod` objects and records helper callers,
    - computes per-file helper counts (a helper is attributed to the file it's **defined in**), the project summary, and the helper-usage table.

This is where analyzer facts + line metrics are *merged*.

1. **`models.py`** — frozen dataclasses: `LineCounts`, `FileReport`, `TcfMethod`, `HelperRef`, `HelperUsage`, `ProjectSummary`, `AnalysisResult`. The typed shape the report and the web layer consume.

1. **`report.py`** — `render_text(result)` (the `.txt` report, in the required section order), `render_json(result)` (the structured `.json` export), and the source-tree builders (`build_source_tree` for text, `build_tree_data` for the HTML tree).

1. **`web/server.py`** — `create_app()` wires routes:
    - `GET /` — the form (prefilled from the saved last-folder state).
    - `POST /analyze` — run an analysis, render `results.html`; on error re-render the form with a message. Builds the JSON blobs (`tcf_data`, `helper_data`) for client-side pagination.
    - `GET /pick-folder` — opens a native folder dialog **on this machine** (Tkinter); returns JSON; degrades gracefully if there's no GUI.
    - `before_request` auth guard, `after_request` `Cache-Control: no-store`, friendly 403 page.

1. **`__main__.py`** — chooses a free port (next open if 5000 is busy), starts Flask on `127.0.0.1`, prints + opens the tokenized URL. Host is **hard-coded** to localhost.

---

5. The web UI (`web/static/`, `web/templates/`)

- **All behaviour is in `app.js`** — one IIFE, vanilla JS, no build step. Notable parts: the loading overlay (with BFCache reset), expand/collapse tree, `.txt`/`.json`/CSV downloads (built in the browser), sortable tables, the native folder-picker call, and the generic `createPaginator(cfg)` used for both the TCF-method and helper-usage lists.
- **`style.css`** — hand-written, no external fonts/assets (local-only). Watch specificity (see §9, the

`[hidden]` gotcha).

- **Templates** — `base.html` (shell + overlay + footer), `index.html` (form), `results.html` (the five report sections + JSON data blobs). The source tree uses native `<details>`/`<summary>` so collapsing needs no JS.

---

6. The C# analyzer (`csharp-analyzer/src/TcfAnalyzer/`)

Flow per run: `Program.cs` reads the whole stdin as JSON → `ProjectAnalyzer.Analyze` → one `CSharpCompilation` over all files → `FileAnalyzer.Analyze` per file → JSON to stdout.

- **`Program.cs`** — I/O only: deserialize request, call `ProjectAnalyzer`, serialize response. Keep logic out of here.
- **`ProjectAnalyzer.cs`** — parses every file into a `SyntaxTree`, builds the **single** compilation with BCL refs, then analyzes each file inside a `try/catch` so one bad file can't abort the others (it yields a flagged, empty `FileResult`).
- **`FileAnalyzer.cs`** — per file: walk `MethodDeclarationSyntax`, mark `isTcf` by prefix, compute complexity, and for TCF methods call `HelperResolver`. Also collects comment spans and a syntactic error count.
- **`HelperResolver.cs`** — *the core.* For each invocation inside a TCF method it asks the semantic model for the called symbol and keeps it **iff** it is an ordinary method, **defined in source** (not metadata/BCL), and **non-TCF**. See §9 for the overload-resolution subtlety this file handles.
- **`ComplexityCalculator.cs`** — cyclomatic complexity (base 1 + branches/loops/cases/ `&&`/`||`/`?:`).
- **`CommentCollector.cs`** — comment trivia → spans for the Python line counter.
- **`ReferenceLoader.cs`** — BCL metadata references from the runtime TPA list.
- **`Models.cs` / `Json.cs`** — request/response records; `System.Text.Json` with **camelCase** naming (so C# `IsTcf` ↔ JSON `isTcf` ↔ Python `"isTcf"`).

---

7. "I want to change X" — where to look

Goal	Touch
Change the **default prefix**	`web/server.py` `DEFAULT_PREFIX`
Change **what counts**	`csharp-analyzer/.../HelperResolver.cs` (+ a test in `AnalyzerTests.cs`)

Goal	Touch
as a helper	
Change **line-counting rules**	`line_metrics.py` (+ `tests/python/test_line_metrics.py`)
Add a **new metric per TCF method**	C#: emit it in `FileAnalyzer`/`Models`; Python: read it in `orchestrator._analyze_csharp`, add to `models.TcfMethod`, surface in `report.py` + `server.py` `tcf_data` + `results.html`/`app.js`
Change **cyclomatic complexity**	`ComplexityCalculator.cs` (+ test)
Add/adjust a **report section or its order**	`report.py` (`render_text`/`render_json`) and `templates/results.html`
Change **C/C++ comment handling**	`cpp_comments.py` (+ `test_cpp_comments.py`)
Add a **file extension**	`filetypes.py` (only if explicitly required)
Change **UI behaviour**	`web/static/app.js`; styling in `style.css`; markup in `templates/`
Change the **Python ↔ C# JSON shape**	both `Models.cs`/`Json.cs` **and** the Python reader (`analyzer_bridge.py`/`orchestrator.py`) — they must move together
Change **packaging/delivery**	`packaging/codeanalyzer.spec`, `scripts/package*.{sh,ps1}`

Rule of thumb: a change that affects **C# semantics** starts in `csharp-analyzer/`; a change to **counting, reporting, or UI** starts in Python/`web`. If it crosses the JSON boundary, update **both** sides and a test.

---

8. Build, run, test, package

```
# One-time Python env
~/.local/bin/uv venv .venv --python 3.12
~/.local/bin/uv pip install -e ".[dev]" --python .venv/bin/python

# Build the C# analyzer for local dev, then make it discoverable
scripts/build-analyzer.sh linux-x64     # -> artifacts/analyzer/linux-x64/analyzer
```

```
scripts/stage-analyzer.sh  linux-x64     # copies it to src/codeanalyzer/_bundled/

# Run the app  ->  http://127.0.0.1:5000/?t=<token>
scripts/run-dev.sh                        # or: .venv/bin/python -m codeanalyzer

# Tests (Python + C#)
scripts/run-tests.sh
# or individually:
PYTHONPATH= PYTEST_DISABLE_PLUGIN_AUTOLOAD=1 .venv/bin/python -m pytest tests/python -q
dotnet test csharp-analyzer/TcfAnalyzer.sln

# Lint / types
.venv/bin/ruff check src tests
.venv/bin/mypy
```

The bundled binary is gitignored. After any change to the C# code you must **rebuild + restage** (the two `*-analyzer.sh` steps) for Python to pick it up — and for a Windows release you must rebuild on Windows.

Windows delivery (on a Windows box with .NET 8 SDK + Python 3.12):

```
.venv\Scripts\pip install -e ".[dev,package]"
powershell -ExecutionPolicy Bypass -File scripts\package-windows.ps1
# -> dist\codeanalyzer\codeanalyzer.exe  (ship the whole dist\codeanalyzer\ folder)
```

PyInstaller does not cross-compile: the Windows exe must be built on Windows.

Quality bar before committing: `ruff` clean, `mypy` clean, all Python + C# tests green. A rename or signature change often reshuffles imports — run `ruff check --fix`.

---

9. Gotchas & hard-won lessons

- **Helper calls with non-trivial arguments (the big one).** Under the ad-hoc compilation, an argument like `Log("line " + n)` or `Log(x.ToString())` can leave the *call's* `Symbol` **null** with `CandidateReason == OverloadResolutionFailure` — even though the target method is obvious. The naive check (`Symbol is IMethodSymbol`) then drops the helper. `HelperResolver` therefore **falls back to `CandidateSymbols`** when `Symbol` is null, applying the same in-source/ordinary/non-TCF filters (so a same-named BCL call is still excluded). If you ever rewrite the resolver, **keep this fallback** and keep its two regression tests. Note `tree.GetDiagnostics()` reports only *syntax* errors, so these binding quirks don't show up as `hasErrors`.
- **`[hidden]` and CSS specificity.** Setting `element.hidden = true` only hides via the UA rule `[hidden]{display:none}`, which a more specific selector (e.g. an id with `display:flex`) silently overrides. If you give a hideable element a `display` in CSS, also add an explicit `…[hidden]`

`{display:none}` rule. (Bit us on the loading overlay and the pagers.)

- **Headless folder picker.** `/pick-folder` opens a *blocking* native dialog. Under `TESTING` it short-circuits to "unavailable" so tests don't hang; on a real headless box the Tk import/dialog fails and the UI falls back to manual entry. Don't call it in automated flows.
- **JSON key casing.** The contract is camelCase via `System.Text.Json`. A C# property `FooBar` becomes `fooBar` on the wire and Python must read `"fooBar"`. Rename both ends together.
- **pytest on this dev box.** A stray system Python/ROS install can leak plugins; the test script clears `PYTHONPATH` and sets `PYTEST_DISABLE_PLUGIN_AUTOLOAD=1`. Use `scripts/run-tests.sh`.
- **Determinism.** The scanner sorts directories and files; keep new traversal/ordering deterministic so reports diff cleanly.

---

10. Invariants you must not break

These come straight from the requirements spec — read it before any behavioural change.

- **Local-only.** No network requests, no cloud/AI APIs, no telemetry, no remote logging, at runtime, in *any* component (including the bundled analyzer). The web server binds to `127.0.0.1` only. If you add a dependency, verify it phones home to nobody.
- **Helpers are semantic, never name-based.** A file or method containing "helper" means nothing. A non-TCF method that no TCF method calls is **not** a helper. Library/ framework calls are **not** helpers.
- **No TCF-to-TCF report.** If a TCF method calls another TCF method, it is ignored — not reported as a dependency.
- **C/C++ get file-level metrics only**, with `TCF count = 0` and `helper count = 0`.
- **Don't crash on bad input.** Invalid folder, empty folder, no supported files, encoding errors, syntax errors — all produce understandable warnings/errors, and a bad file never stops analysis of the others.
- **Report order is fixed:** Project Summary → File Summary → Source Tree → TCF Method Details → Helper Usage Summary.
- **Scope discipline.** Don't add file types, cloud/AI analysis, or telemetry unless explicitly asked. When a requirement is ambiguous, choose the stricter reading.

---

11. Where to start reading

If you are new and want the shortest path to understanding:

1. The requirements spec (project root) — what the tool must do.
2. This guide §2 (why) and §3 (map).
3. `orchestrator.py` — the whole Python pipeline in one file.
4. `HelperResolver.cs` — the one piece of real cleverness.
5. Run `scripts/run-dev.sh`, analyze `examples/edge-cases`, and read its annotated files (`examples/README.md` documents the expected ground truth).